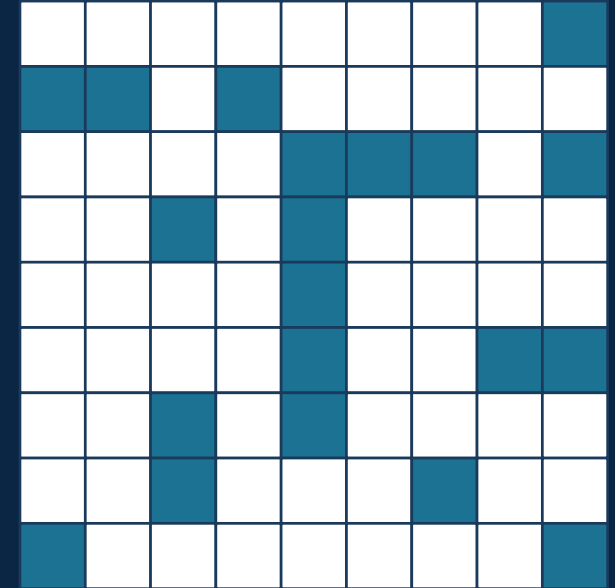


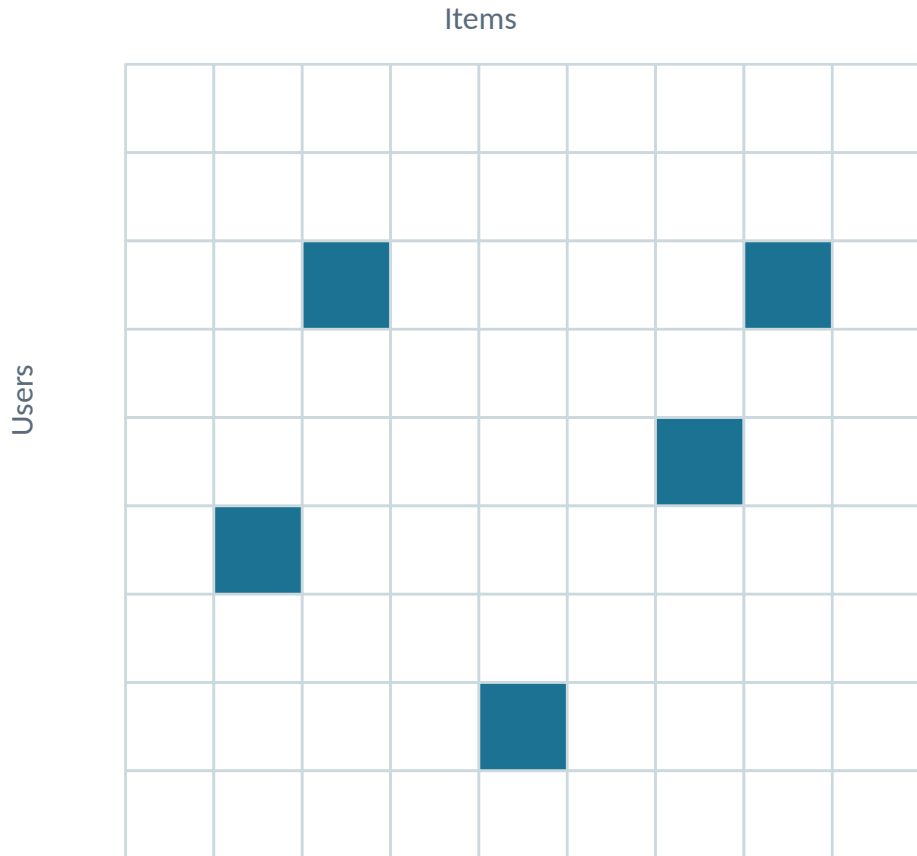
Distributed Stochastic Gradient Descent for Matrix Factorization

Scaling web-scale recommender systems with stratified SGD (DSGD)

Yagel Alfasi

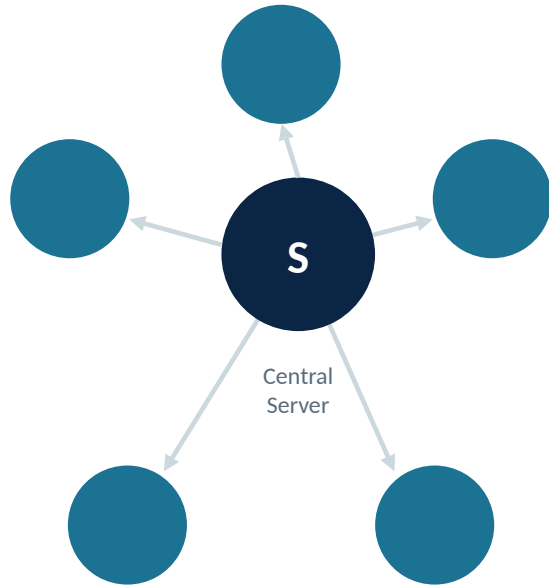


The Web-Scale Challenge



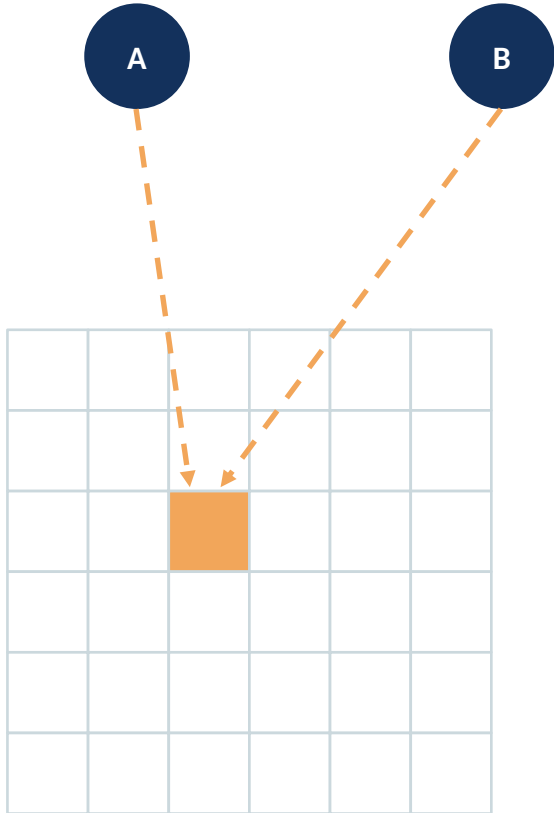
- Massive dimensions: matrices with millions of rows (users) and millions of columns (items).
- Extreme sparsity: billions of non-zero entries (ratings, transactions, clicks) scattered across a mostly empty matrix.
- Goal: recover latent factors that predict the missing entries e.g. recommender systems.
- Computational need: split the work across many machines without the machines corrupting each other's updates.

The Baseline: Centralized Distributed Gradient Descent



- Standard distributed optimization (DGD) relies on a central coordinating server.
- Compute nodes calculate local gradients and transmit them back to the center every step.
- The server aggregates the updates and broadcasts new parameters.
- Result: communication overhead dominates computation, and the server is both a synchronization delay and a single point of failure.

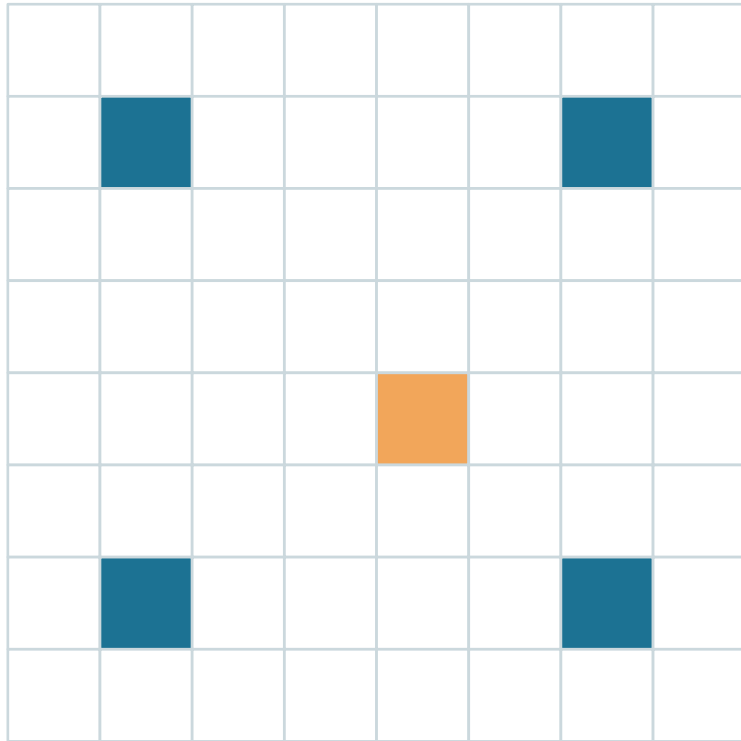
The Update-Collision Problem



collision at shared row/column

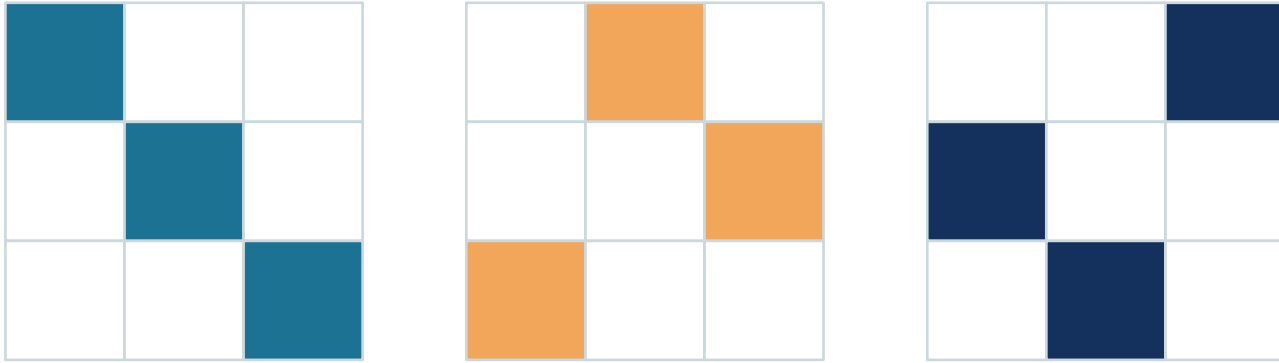
- SGD refines parameters sequentially: step $n+1$ strictly requires the output of step n .
- Updating one matrix entry modifies its entire row factor and column factor.
- If two nodes update the same row or column at the same time, their writes collide and overwrite each other.
- Naive parallel SGD therefore corrupts convergence - parallelism needs a structural safeguard, not just more machines.

The Structural Fix: Interchangeability



- Definition: two training points are interchangeable if they share neither a row nor a column.
- Theorem :running SGD on interchangeable points in any order, including in parallel, gives the exact same result as running them sequentially.
- This structural property of the matrix, not any special assumption about the loss function, is what licenses safe parallelism.

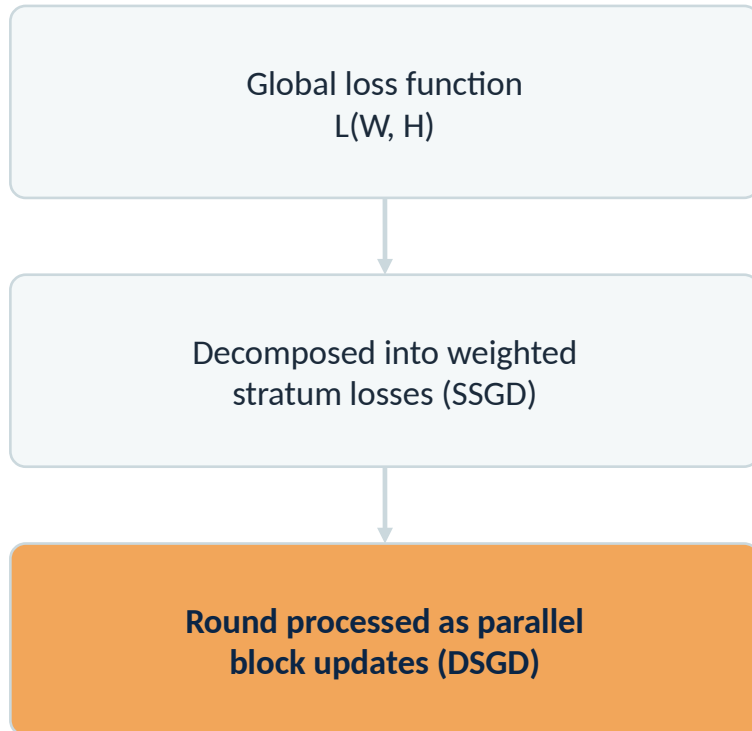
Round: Diagonals of Independent Blocks



Three of the $d!$ possible Round for a 3×3 block grid

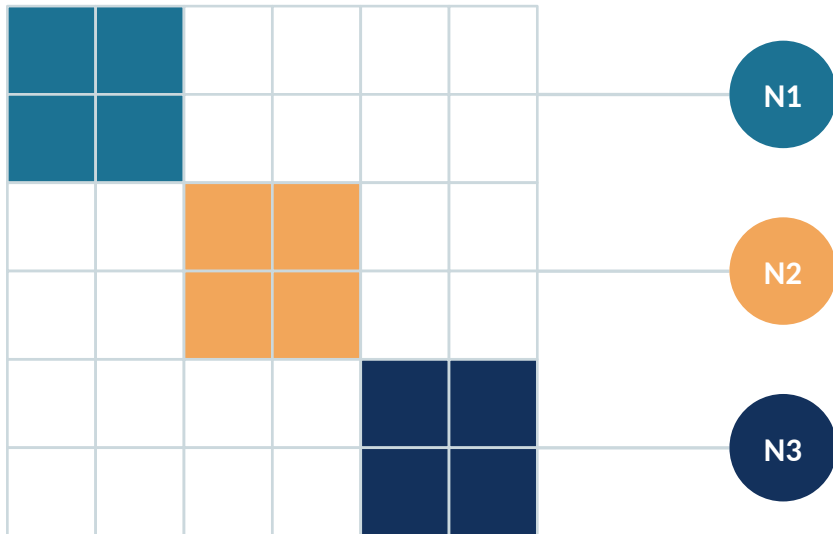
- Divide the input matrix into a $d \times d$ grid of blocks.
- A stratum groups d blocks that pairwise share no row and no column (a “ d -monomial” set).
- Blocks within one stratum are automatically interchangeable, so each can go to a different compute node.
- d such disjoint Round are enough to cover the entire training matrix once.

From Theory (SSGD) to Practice (DSGD)



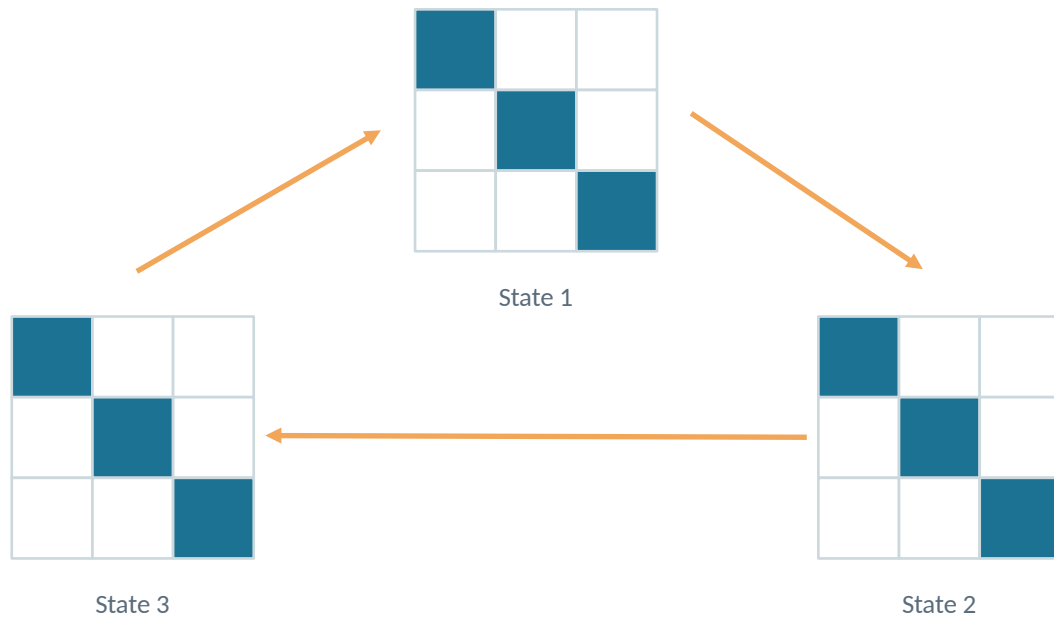
- SSGD (stratified SGD): a general recipe - take a downhill step on one stratum's loss at a time, switching Round according to their weights.
- Convergence is proven via stochastic approximation theory and regenerative process theory, under conditions on step sizes and how Round are sequenced.
- As defined, SSGD is still one sequential update sequence.
- **DSGD's contribution:** process all blocks in a chosen stratum in parallel, and prove this is equivalent to some valid sequential SSGD run - so the same convergence guarantee carries over.

The DSGD Architecture



- Both the training data and the factor matrices W , H are partitioned into blocks - low memory footprint per node.
- Each node permanently holds one row-block of data plus the matching factor block.
- Within a stratum's compute step, nodes update independently - no communication is needed during that step.
- Between subepochs, nodes only exchange small factor-matrix blocks, not raw data but this exchange is real network traffic, and in the paper's Hadoop experiments it was the dominant cost of DSGD's wall-clock time.

Subepochs and Epochs



- Subepoch: every node processes its assigned block of the current stratum, independently and in parallel.
- Transition: instead of moving the raw dataset, nodes pass compact factor blocks to the next node in the rotation.
- Epoch: cycle through all d Rounds once - roughly equivalent to one full pass over the training set.
- Per-epoch cost differs by algorithm: on the paper's Netflix test, DSGD ran ~43 epochs, DGD ~61, PSGD ~30, and ALS ~10 epochs within the first hour.

Why DSGD Generalizes

Specialized algorithms (e.g., ALS)

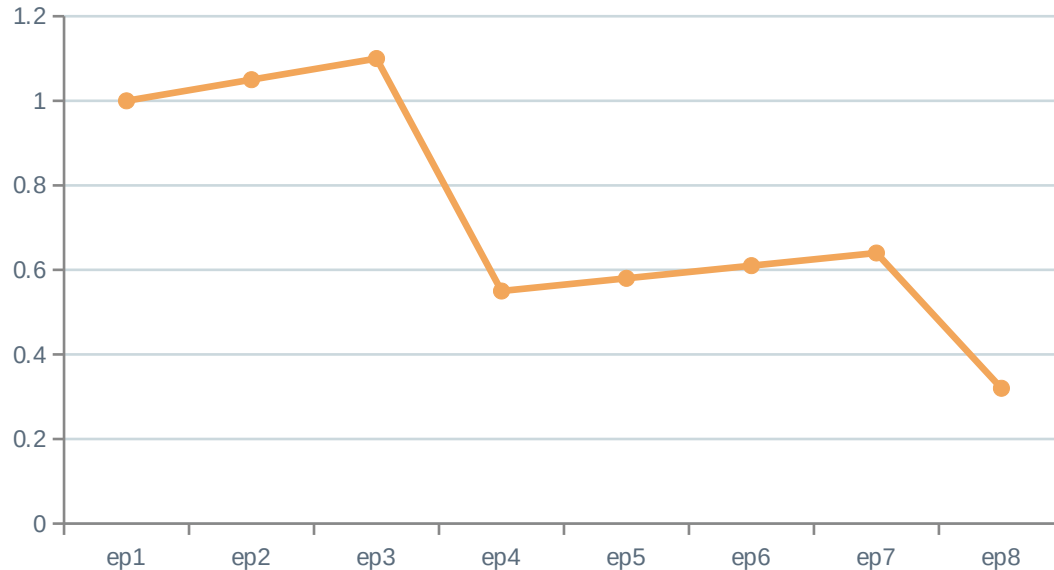
- Built for one specific loss function.
- ALS only handles quadratic (squared) loss.
- A new loss function means a new algorithm derivation.

DSGD (generic)

- Works for any loss expressible as a sum of local losses.
- Covers squared loss, generalized KL-divergence, L1/L2-regularized loss, and nonnegativity constraints.
- One algorithm, many recommender-system objectives.

Tuning in Practice: Adaptive Step Size

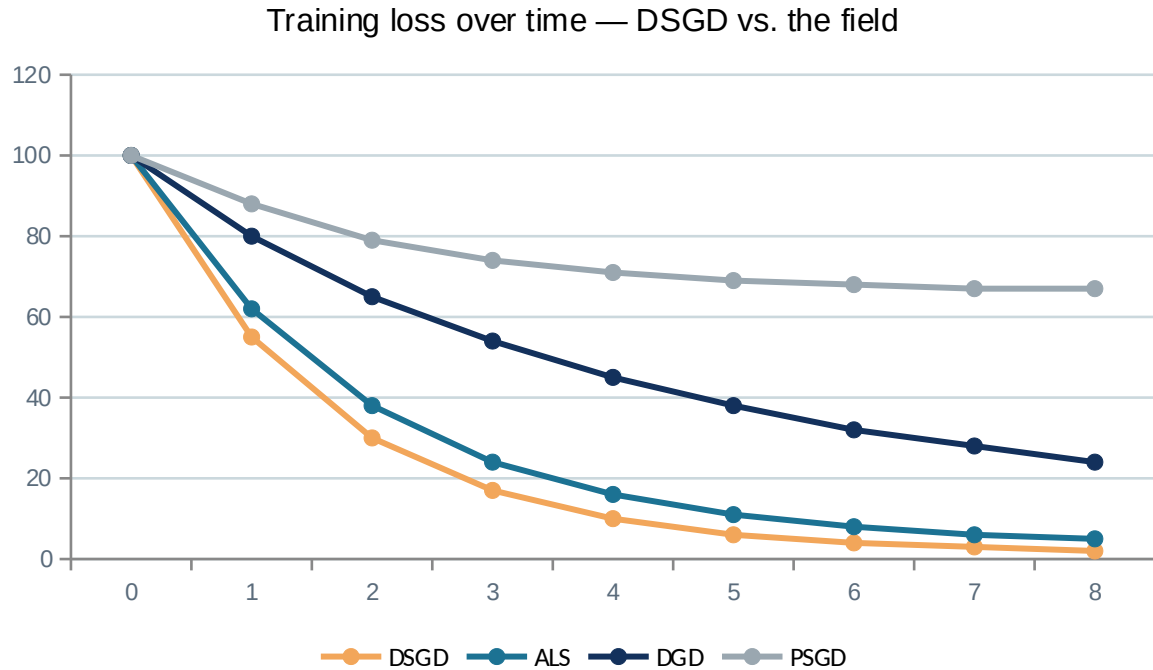
Illustrative “bold driver” step-size trace



Step size rises steadily, then is cut sharply whenever loss increases.

- Starting step size: try several candidates (1, 1/2, 1/4, ...) in parallel on a small (~0.1%) data sample; pick the best-performing one.
- “Bold driver” heuristic during training: increase the step size by ~5% whenever loss decreases over an epoch; cut it by ~50% whenever loss increases.
- This removes the need to hand-tune a fixed learning-rate schedule for web-scale runs, where a full loss evaluation is expensive.
- The paper flags this as one of its practical contributions - distinct from, but essential alongside, the theoretical convergence guarantee.

Convergence & Scalability — The Bottom Line



DSGD converges fastest of every method tested.

- DSGD wins on speed - it out-converges ALS, DGD, and PSGD in the paper's own experiments.
- DSGD wins on scale - tested up to 572GB and 25.6 billion ratings, with only a modest increase in runtime.
- **The catch:** overhead doesn't disappear, it just moves. At large enough cluster sizes, network and coordination costs - not the algorithm - set the ceiling.

Thank You

Questions?

